

CS313 Deep Learning for Artificial Intelligence — Final Project Report

Time-Series Data and Application to Stock Markets

Student ID: 240069

Course: CS313 Deep Learning for Artificial Intelligence, Spring 2026

Fulbright University Vietnam

GitHub: <https://github.com/tonasthesecond/cs313-stock-analysis>

Introduction

This project applies deep learning to stock price forecasting across two datasets — Nasdaq and Vietnamese stocks — and extends into trading signal classification, portfolio construction, and model deployment. The tasks build on each other: the architecture selected in Task 2 drives Tasks 3 and 4, and the Task 4 models are what gets served by the API in Task 5.

Shared Methodology

Data Split

All splits are strictly chronological: 64% training, 16% validation, 20% test. `shuffle=False` is enforced at every stage. The test set is always the most recent observations and is never used during training or model selection. Shuffling time-series data would let the model train on future observations and evaluate on past ones, producing invalid results. The validation set is used for early stopping and, in the architecture sweep, for comparing configurations.

Cross-Validation

Only time-series-aware cross-validation is used: an expanding window where the training set grows forward in time and validation always follows it. Standard k-fold is not used because it

randomly mixes past and future samples across folds.

Per-Window Normalisation

Each sliding window is normalised independently using per-feature min-max scaling. Targets are normalised against the same window statistics and denormalised after inference. Global normalisation was avoided because it leaks future price statistics into the training distribution. Windows where the Close range is less than 0.1% of the mean price are filtered out — these produce near-zero denominators and cause numerical instability.

Task 1 — Nasdaq Stock Price Prediction

Data

All six OHLCAV columns are used: Open, High, Low, Close, Adjusted Close, Volume. The demo used only Open. Window size is 30 days. One company CSV is loaded as the representative sample for Tasks 1.1–1.3. The same model architecture and training pipeline is reused across all three subtasks — only the output target changes.

Architecture

Three Conv1D stages ($64 \rightarrow 128 \rightarrow 64$ filters) each followed by MaxPooling1D, then Dense(100) $\rightarrow$ linear output. Training: Adam lr=1e-2, early stopping patience=10, batch size 64. Models are cached to disk and reloaded on subsequent runs.

Subtasks

- **1.1:** Predicts next-day Close.
- **1.2:** Predicts Close at day +3 (`FORECAST_DAY = 3`). Same architecture, target index shifts.
- **1.3:** Predicts 3 consecutive Close prices (`FORECAST_DAYS = 3`). Output layer widened to 3; loss is MSE averaged across all output steps.

RMSE and MAE are reported per output day. Error grows with forecast horizon.

Task 2 — Vietnam Stock Price Prediction

Company Filtering

Companies must have at least 160 raw rows, all OHLCV columns present without nulls, and mean volume above 1. This gives roughly 120 usable data points after indicator warm-up. Companies with less data or gaps were excluded.

Feature Engineering

Five technical indicators are added to OHLCV, giving 11 features total:

- **SMA5, SMA20** — short and medium-term moving averages
- **RSI14** — momentum oscillator, bounded $[0, 100]$
- **MACD, MACD_signal** — difference between 12-day and 26-day EMAs, plus its 9-day signal line
- **Volatility** — 10-day rolling standard deviation of daily returns

On additional Vietnam data (dividends, financial ratios, industry analysis): These were not included. Financial ratios are quarterly and slow-moving — appending them to a daily-horizon model without careful alignment would likely add noise rather than signal. Dividend history could be useful for flagging ex-dividend price drops, but that was out of scope. The 11 features above were sufficient to run a reasonable experiment.

Architecture Search

Five architectures were benchmarked at window=30, then the top three (v0, v1, v2) were swept across window sizes 10, 20, 30, 60 — 12 configurations total.

Name	Description
v0	Stacked LSTM(128 $\rightarrow$ 64)

Name	Description
v1	Conv1D(64→128) + MaxPool + LSTM(64)
v2	2×Conv(64)→Pool, 2×Conv(128)→Pool, LSTM(128), Dropout(0.3), Dense(64)
v3	CNN as v1, stacked LSTM(64→32)
v4	CNN + LSTM + temporal attention

v2 at window=20 had the lowest RMSE and MAE and was used for all subsequent Vietnam tasks. v4 did not outperform v2. No strong explanation is claimed for why — these results came out of the sweep.

The benchmark at window=30 was run first to narrow down the architecture candidates before committing to the full sweep. All five architectures were trained for 30 epochs with Adam lr=3e-4 and early stopping patience=10. The same company CSV and the same train/val/test split were used across all configurations so the comparison was consistent. v0 (pure LSTM) was the weakest. v2 and v4 were close, with v2 consistently ahead. The window sweep then confirmed window=20 as better than 30 or 60 for v2 specifically. Training on window=10 was unstable for most architectures — not enough context.

Subtasks 2.1, 2.2, 2.3

Same structure as Task 1. v2 at window=20, 11 features, prices in VND.

Task 3 — Trading Signal Identification

Problem Setup

Tasks 3.1 and 3.2 predict whether a given moment is a good time to buy or sell, outputting a probability in $[0, 1]$. Two separate classifiers are trained: one for buy signals, one for sell. They are trained independently because their label distributions differ.

Labelling

A window of 10 days before and 10 days after each timestep is used to generate labels:

- **Buy = 1** if the current Close $\leq$ 20th percentile of that 20-day window
- **Sell = 1** if the current Close $\geq$ 80th percentile

The future half of the window is only used at label generation time. The model at inference sees only the past 10 steps.

The 20th/80th percentile threshold is a design choice. A tighter threshold (e.g. 10th/90th) produces fewer but higher-confidence labels — harder to learn from, fewer training examples of the target class. A looser threshold produces more labels but they're noisier, marking timesteps that aren't clearly local extrema. 20/80 was a reasonable middle ground; it wasn't tuned further.

On manual feature engineering for signals: The same 11-feature set from Task 2 is used. RSI and MACD are already encoding the kind of momentum information the labels are trying to capture, so feeding them in directly makes more sense than asking the model to derive that from raw prices.

Architecture and Class Balancing

Both classifiers: LSTM(128, return_sequences=True) $\rightarrow$ Dropout(0.2) $\rightarrow$ LSTM(64) $\rightarrow$ Dropout(0.2) $\rightarrow$ Dense(32, relu) $\rightarrow$ Dense(1, sigmoid). With 20th/80th percentile thresholds, about 20% of labels are positive. Class weights from `compute_class_weight('balanced')` are applied so the model doesn't just predict the majority class. Evaluation uses precision, recall, and F1 — recall on the positive class is prioritised since missing a real signal is more costly than a false alarm.

Task 4 — Portfolio Composition, Risk Management, and Optimisation

Company Filtering

Same criteria as Task 2: 160 raw rows minimum, no null OHLCV, mean volume > 1.

Task 4.1 — Profitable Stock Selection

A v2 model is trained per company. Predicted return is computed from the final 20-day window:

```
predicted_return = (predicted_close - last_actual_close) / last_actual_close
```

Companies are ranked by this value. Only those with positive predicted return are considered as portfolio candidates.

Task 4.2 — Risk Management

Risk score per company:

```
risk_score = volatility + |max_drawdown|
```

Volatility is the standard deviation of daily returns. Max drawdown is the worst peak-to-trough decline over the full price history. The two are combined because they capture different kinds of risk: a company can have moderate day-to-day volatility but still have crashed 70% once years ago. Volatility alone wouldn't flag that. Max drawdown alone would penalise companies that recovered. The additive combination is simple and interpretable — no weighting was applied between the two components. Companies above the 70th percentile of this score are excluded. A percentile threshold rather than an absolute cutoff means the filter scales with the volatility of the current universe.

Task 4.3 — Portfolio Composition

Portfolio candidates are companies with positive predicted return and risk score below the threshold.

Equal-weight: every candidate gets weight $1/N$. Doesn't concentrate on the model's top picks — treats all filtered candidates equally.

Return-weighted: each company weighted proportionally to its predicted return, normalised to sum to 1. A median split further divides this into an aggressive group (above median predicted return, return-weighted) and a conservative group (below median, equal-weighted). This gives a practical way to split the same candidate set into two portfolios for different risk tolerances without changing the filtering logic.

Task 5 — Model Deployment and SaaS Interface

Task 5.1 — FastAPI REST API

Models are served via `stock_api.py`, a FastAPI application with four endpoints:

Method	Path	Description
GET	<code>/health</code>	Returns <code>{"status": "ok"}</code>
GET	<code>/tickers</code>	Lists tickers with a deployed model
GET	<code>/window/{ticker}</code>	Returns the latest 20-row input window and last Close
POST	<code>/predict</code>	Runs inference, returns predicted Close (VND)

How to start the server:

Open the notebook in Colab and run `Runtime → Run all`. The setup cell clones the repo, the server cell starts uvicorn in a background thread, and the tunnel cell polls until a `trycloudflare.com` URL appears (typically within 15 seconds). If the tunnel cell fails, rerun it — the free Cloudflare tier occasionally needs a second attempt.

To run standalone:

```
cd cs313-stock-analysis
pip install fastapi uvicorn
uvicorn stock_api:app --host 0.0.0.0 --port 8000
```

How to send a prediction request:

```
# get the input window
```

```
curl http://localhost:8000/window/ADG-VNINDEX-History
```

```
{
  "window": [[24300.0, 24500.0, 24100.0, 24250.0, 150000.0, 24100.2, 24198.4, 52.3, -80.0],
  "last_close": 24250.0
}
```

```
# predict
```

```
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"ticker": "ADG-VNINDEX-History", "window": [...], "close_idx": 3}'

{
  "ticker": "ADG-VNINDEX-History",
  "predicted_close": 24580.0
}
```

Task 5.2 — Web Interface

The application serves an HTML/CSS/JS frontend at `GET /`. Once the tunnel URL is printed, open it in a browser. Select a company from the dropdown, click predict, and the result shows the predicted next-day Close in VND and percentage change from the last known price. The frontend calls `/window/{ticker}` then `/predict` — all data preparation happens server-side.

Limitations

Results are single-company. Tasks 1 and 2 run on one company CSV at a time for demonstration. The architecture sweep used one Vietnamese company. Whether the chosen architecture and window size generalise across the full universe wasn't tested.

Predicted return as a selection signal is weak. Using a one-step-ahead price prediction as a buy signal is a reasonable starting point, but the model isn't trained to maximise return — it's trained to minimise MSE on the next Close. A company with low prediction error might still have a low predicted return. The ranking in Task 4.1 is a proxy, not a direct profitability signal.

No backtest. The portfolio construction logic produces allocations but doesn't simulate actual trading returns over time. There's no way to tell from this project whether the selected portfolios would have outperformed a simple benchmark.

Signal classifiers aren't evaluated end-to-end. The buy and sell classifiers output probabilities, but those probabilities aren't combined or tested in a trading strategy. The output of Task 3 feeds nothing downstream — it stands alone.

Conclusions

Architecture. v2 (paired CNN blocks + LSTM128) won the benchmark and sweep on Vietnamese data across 12 configurations. The result came from the sweep.

Feature engineering. Adding SMA, RSI, MACD, and volatility to OHLCV helped, particularly for the signal classifiers where those features directly relate to how the labels are defined.

Window size. 20 days won the sweep on Vietnamese data. It was treated as a hyperparameter and swept rather than assumed.

Normalisation. Per-window min-max normalisation was necessary for stable training across companies with different price scales.

Deployment. Clone the repo, run all cells, open the URL. Pre-trained models are committed so nothing needs to retrain.

Data pipeline. No shuffling, no look-ahead normalisation, no random k-fold at any point in the project. Getting the split right matters more than architecture choices — a well-tuned model on a leaky split is worthless.